IEEE Open Journal of the Computer Society

Received 6 December 2025; revised 25 January 2026; accepted 8 February 2026. Date of publication 12 February 2026; date of current version 25 February 2026. The review of this article was arranged by Associate Editor Xiang Sun.

Digital Object Identifier 10.1109/OJCS.2026.3664314

MCP-Secure: A Runtime Access Control Layer for Privilege-Aware LLM Agent Tooling

GAMINI SINGH [1] AND VIJAY K. MADISETTI [2] (Fellow, IEEE)

[1]College of Computing, Georgia Institute of Technology, Atlanta, GA 30332 USA
[2]School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA 30332 USA

CORRESPONDING AUTHOR: VIJAY K. MADISETTI (e-mail: vkm@gatech.edu).

ABSTRACT Tool-enabled language-model agents introduce new security risks because their behavior evolves over multi-step workflows, yet existing defenses primarily rely on static allowlists or infrastructure isolation. This article presents MCP-Secure, a lightweight, host-side enforcement layer for the Model Context Protocol (MCP) that applies scoped access, read-only defaults, and approval-gated privilege elevation at runtime. MCP-Secure tracks permissions through a session-level state machine and mediates every tool invocation without modifying agents or MCP servers. We evaluate the framework across 1,080 executions spanning multiple models, tasks, and adversarial simulations. Results show that scoped access alone blocks most unsafe behaviors, read-only enforcement reliably neutralizes all mutating attack vectors, and approval-gated elevation maintains strong safety while enabling controlled write operations. Across configurations, the wrapper also shapes agent planning, reducing unsafe attempts as policies tighten. These findings demonstrate that MCP-Secure provides a practical, reproducible mechanism for enforcing least-privilege constraints in tool-enabled LLM systems, offering strong adversarial resistance with interpretable safety– utility trade-offs.

INDEX TERMS Large language models, AI safety, MCP.

I. INTRODUCTION

Large-language-model (LLM) agents are increasingly being deployed as part of enterprise automation pipelines, where they coordinate multi-step workflows, interact with private systems, and perform actions traditionally reserved for authenticated users. Recent surveys emphasize that this shift to active tool use sharply increases the security stakes. LLMs inherit classical software vulnerabilities while simultaneously exhibiting unique failure modes arising from multi-step planning and autonomous tool usage [1], [2]. These risks are amplified when LLMs acquire the ability to invoke external tools, APIs, and data services.

This problem becomes particularly acute by the emergence of the Model Context Protocol (MCP), which has rapidly evolved into a unified mechanism for enabling structured tool calls, external data access, and multi-server interoperability [3]. While MCP significantly enlarges an agent's operational perimeter, it also enlarges the potential attack surface. Real incidents have demonstrated the severity of these risks. A misconfigured GitHub MCP server allowed a crafted issue to leak private repository contents [4], while a production MCP integration in Asana triggered cross-tenant data exposure due to insufficient isolation [5]. These failures illustrate how a single unsafe tool call, whether hallucinated, misclassified, or injected can cause cascading security consequences.

Recent empirical studies reinforce this concern. Tool-using LLM agents display brittle behavior under adversarial or ambiguous conditions, with multi-step reasoning sequences producing unsafe calls that do not appear in single-step prompting [6]. Broader security reviews similarly note that existing defenses for LLMs do not adequately capture the dynamic, stateful and interaction-driven nature of tool invocation, where failures often emerge only after chains of tool responses, contextual shifts, or interdependent API effects [7]. Surveys of LLM-based tool agents highlight that real-world deployments often rely on permissive default scopes, sparse auditing, and weak separation between read-only and mutating operations. This leads to conditions under which small misjudgments in model reasoning can escalate into unsafe system-level actions [8]. Collectively, these findings highlight the need for runtime mechanisms capable of regulating agent

© 2026 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

behavior over entire execution traces rather than at isolated prompts or static configuration boundaries.

In practice, current MCP deployments operate in complex, multi-step workflows with overly broad access. They lack safeguards to stop attacks such as prompt injection, tool misuse, and bloated permissions [9]. Traditional security measures, such as container isolation, OS sandbox profiles, or static allow-lists, partially mitigate this risk but impose heavy operational overhead or require invasive architectural changes [10]. They also fail to capture the temporal dimension of agent behavior, where unsafe actions might only surface after a series of misinterpreted tool responses, chained server calls, or ambiguous reasoning steps. Although lightweight runtime-level enforcement has been proposed as a promising direction, most existing approaches remain either model-specific, limited to prototype settings, or too coarse to support granular, capability-aware control [11].

To address these gaps, we introduce **MCP-Secure**, a general-purpose host-side enforcement wrapper designed to regulate LLM agent tool use under least-privilege principles. Instead of modifying agent prompts, altering model internals, or rewriting MCP servers, MCP-Secure intercepts actions at the host boundary, which is the exact point where unsafe behavior manifests. It mediates MCP agent behavior based on three complementary controls: (1) **scoped access**, which limits agents to only the tools and servers required for a task; (2) **read-only defaults**, which treat mutating operations as opt-in rather than default-enabled; and (3) **approval gating**, which introduces explicit checks for sensitive or destructive actions. This design enables fine-grained, runtime-aware enforcement without disrupting existing MCP deployments.

We evaluate MCP-Secure by examining end-to-end agent behavior rather than isolated tool calls. We adopt Salesforce AI Research's MCP-Universe [12], one of the few benchmarks involving real MCP servers and multi-step workflows where unsafe behavior emerges organically. Within this environment, we ground our study in the repository-management domain, which naturally involves high-impact read/write operations. Instead of superficial cross-domain coverage, we perform a deep, high-resolution analysis on two representative tasks: repository creation and branching, and issue search with CSV output. To stress-test enforcement, we run each task across three heterogeneous LLMs (GPT-5, Claude Sonnet 4, Grok-4), three adversarial attack simulations (Privilege Escalation, Malicious Tool Metadata Injection, Multi-Tool Confusion/Chaining), and four enforcement levels (L0–L3). This yields a broad and statistically meaningful perspective on safety–utility tradeoffs in realistic agent workflows.

Contributions: This article makes the following contributions:

- We propose MCP-Secure, a lightweight host-side enforcement architecture that mediates MCP tool invocations at runtime without modifying agents, models, or MCP servers.
- We design and implement a unified least-privilege control mechanism combining scoped access, read-only defaults, and approval-gated privilege elevation, coordinated through a session-level security state machine.
- We present a systematic empirical evaluation across multiple tasks, heterogeneous LLMs, and adversarial attack simulations, enabling reproducible analysis of safety–utility trade-offs.
- We show that runtime access control not only blocks unsafe tool use but also influences agent planning behavior, with unsafe attempts decreasing as enforcement tightens.

II. EXISTING WORK

Research on securing tool-enabled LLM agents has expanded rapidly as these systems transition from prototype demonstrations to enterprise automation pipelines that interact with external tools. Early foundational agent frameworks such as ReAct [13], Reflexion [14], and Toolformer [15] established the viability of multi-step reasoning with external tools, but subsequent analyses revealed that tool-enabled reasoning is brittle and prone to semantic drift, compounding errors, and unintended actions. Broader security surveys have documented that the introduction of tool access magnifies traditional LLM risks such as prompt injection and data leakage, while introducing new failure modes tied to long-horizon, stateful tool use [1], [2]. These results collectively motivate the need for runtime safeguards that operate beyond static prompt filtering or model-level alignment.

Against this backdrop, Model Context Protocol (MCP) has quickly become "the de facto standard" for tool-enabled AI agents. Its flexibility has accelerated real-world adoption, but it has also introduced new security concerns. Empirical studies by Hasan et al. show that thousands of open-source MCP servers grant overly broad or unrestricted host access, with common misconfigurations enabling privilege escalation or unintended file-system exposure [16]. Complementary analyses by Hou et al. map the threat landscape, identifying attack vectors such as multi-tool chaining, semantic ambiguity in tool descriptions, and unsafe state carryover across turns, ultimately calling for finer-grained privilege control and session-aware safeguards [9]. Enterprise-oriented guidance similarly warns that session-oriented, dynamic nature of MCP, challenges traditional authentication and sandboxing strategies, necessitating new forms of runtime oversight [10]. These works stress that traditional software security measures are inadequate, and purpose-built enforcement is required.

Benchmarking work further highlights where failures arise in practice. Guo et al. (2025) introduce MCP-LIB, a framework for evaluating agents under adversarially perturbed tool environments, showing that even aligned models frequently misinterpret ambiguous tool responses or escalate privileges unintentionally [17]. Microsoft Research's SafeMCP adopts a similar philosophy, focusing on capability bounding and dynamic scope verification, though its implementation details remain proprietary [18]. Together, these systems underscore the need for programmable security layers that intervene at runtime rather than relying solely on static manifests.

To address this, lightweight, host-side and non-invasive wrappers have been proposed to secure LLM agent tool use as alternatives to heavy containerization or OS-level isolation. They intercept an agent's tool invocations at runtime to enforce security policies, without relying solely on in-model prompt guardrails or heavy cloud sandboxes. AgentBound is a framework that AgentBound enforces OS-level resource access declarations for MCP tools, providing low-overhead controls but without governing how permissions evolve over a session [3]. MCP Guardian wraps MCP clients and servers to add authentication, rate-limiting, tracing, and manual gating for certain sensitive actions [19]. Beyond MCP-specific tooling, Wang et al. (2025) propose a domain-specific language for agent safety policy enforcement, enabling developers to express high-level rules over tool use, data handling, and behavioral thresholds [20]. While these efforts meaningfully improve tool-access hygiene, they generally treat each tool call in isolation and do not model the temporal or stateful nature of agent behavior.

Across these lines of work, two consistent limitations emerge. First, existing systems treat security as either static configuration (allowlists, manifests, resource declarations) or as one-shot filtering on individual tool calls. These approaches do not capture temporal properties such as privilege drift, multi-step escalation, or context accumulation across turns. Second, current solutions are fragmented. Different solutions tackle scope checking, metadata validation, sandboxing, or manual gating, but none provide a unified runtime policy engine that tracks an agent's permissions and state evolution throughout a task.

MCP-Secure is designed to address these missing dimensions. Rather than focusing on infrastructure hardening or static configuration, it provides a single, session-aware enforcement layer that scopes tool visibility, enforces read-only defaults, requires explicit approval for sensitive operations, and tracks privilege transitions across the agent's full execution trace. This unification enables fine-grained behavioral control within the agent–tool interaction loop. This is precisely the level of enforcement that recent studies argue is needed but that existing systems do not yet provide. MCP-Secure therefore complements prior work by operationalizing least-privilege principles at runtime, offering a practical and lightweight mechanism to strengthen safety in real-world MCP deployments.

III. PROBLEM STATEMENT AND SOLUTION OVERVIEW

A. PROBLEM STATEMENT

MCP enables LLM agents to invoke tools across multiple servers and domains, but current deployments provide limited mechanisms for regulating how agent privileges evolve over multi-step executions. Permissions are typically defined statically, with weak separation between read-only and mutating operations, allowing unsafe actions to emerge through chained tool use or privilege drift.

Existing defenses primarily operate at fixed boundaries and do not capture the stateful, execution-level behavior of tool-using agents. As a result, there is no systematic way to enforce least-privilege constraints over an agent's full execution trace without modifying agents or MCP servers.

B. SOLUTION OVERVIEW

We address this gap with MCP-Secure, a host-side enforcement wrapper that mediates all MCP tool invocations at runtime. By operating at the agent–tool boundary, MCP-Secure enforces least-privilege constraints without modifying agent prompts, model internals, or MCP servers, allowing it to be deployed as a backward-compatible, drop-in component in existing MCP workflows.

MCP-Secure combines three complementary controls—scoped access, read-only defaults, and approval-gated privilege elevation—coordinated through a session-level security state machine that tracks effective permissions throughout execution. Together, these mechanisms enable dynamic, execution-aware enforcement that adapts to agent behavior over time while remaining model-agnostic and portable across heterogeneous MCP deployments.

IV. PROPOSED SOLUTION

MCP-Secure enforces least-privilege constraints during agent execution by treating each agent run as a stateful session rather than a sequence of independent tool calls. This session-level perspective allows permissions to be evaluated in the context of prior actions, tool responses, and enforcement decisions accumulated over multiple turns.

During execution, each tool invocation issued by the agent is intercepted and evaluated against the current session context. Depending on the requested capability and the agent's effective privileges at that point in execution, an action may be permitted, downgraded to a safe alternative, blocked entirely, or allowed only after temporary elevation. These decisions are not static. They evolve as execution progresses and as the agent's interaction history changes.

A. POLICY CONTROLS

The policy controls in MCP-Secure are designed to regulate agent behavior over complete execution traces, enabling systematic measurement of how least-privilege enforcement affects both safety and task utility. By embedding these controls directly into the runtime execution flow, the framework supports controlled comparison across enforcement levels while preserving realistic agent behavior.

The system is engineered around three complementary policy controls that work together over the course of a session: **scoped access**, **read-only defaults**, and **approval gating**. Each of these controls targets a different class of failure mode reported in recent MCP security analyses, and their combination is intentionally designed to cover gaps that existing approaches do not address.

First, **scoped access** constraints the agent to the minimum set of servers, tools, and capabilities required for a task. In practice, most unsafe behavior arises because agents inherit broad access from default MCP configurations, allowing cross-task tool exposure and unintended privilege surfaces. MCP-Secure formalizes a narrow operational scope for each task and ensures that all calls remain within these boundaries throughout the session. This eliminates cross-domain leakage and significantly reduces the blast radius of misinterpretation, hallucinated tool calls, or misleading tool descriptions.

Second, **read-only defaults** enforce a conservative baseline for interaction with stateful services. While many MCP deployments assume mutating operations are rare or intentional, studies consistently show that agents attempt writes, deletes, and repository-modifying actions even in tasks where they are not required. MCP-Secure treats all mutating effects as opt-in rather than default-enabled. Under this policy, state-changing operations are suppressed unless explicitly permitted, allowing the system to separate out tasks where mutation is essential from those where read-only execution is sufficient.

Third, **approval gating** provides a structured mechanism for safely invoking sensitive or destructive operations. Rather than relying on the agent's internal reasoning to determine whether an action is appropriate, MCP-Secure introduces a dedicated checkpoint that must be passed before certain effects are applied. Importantly, MCP-Secure distinguishes between permanent permissions and temporary elevations. An approval grants time-boxed access to a restricted capability, after which privileges automatically revert. This design mirrors real-world operational controls such as change windows, break-glass access, and temporary escalation policies. In our evaluation, approval decisions are implemented as a deterministic, rule-based policy to ensure reproducibility. This mechanism abstracts either automated organizational policies or human-in-the-loop approval in real deployments.

B. SECURITY STATE MACHINE

While the policy controls describe *what* restrictions MCP-Secure enforces, the security state machine governs *when* and *how* those restrictions apply over the lifetime of a task. Rather than treating permission checks as isolated filters on individual tool calls, MCP-Secure maintains a session-level view of the agent's evolving privilege state. This perspective is essential for understanding and ultimately constraining the temporal patterns through which unsafe behavior tends to emerge.

The heart of this mechanism is a session-level state that records the agent's current privilege posture. Fig. 1 illustrates how enforcement levels transition in response to tool invocations and approval decisions during execution.

At the start of each task, the wrapper assigns the session an initial enforcement level that reflects the organization's desired security posture. The state machine begins in an open, baseline configuration and progressively tightens or relaxes restrictions depending on the enforcement level. In its most permissive form, all declared tools are available with full read/write capabilities, mirroring the behavior of common prototype environments. As the agent proceeds, the state machine interprets each tool invocation in context and the operational surface narrows. Non-essential servers are removed from scope, mutating calls are automatically downgraded to safe read-only behavior, and certain actions become subject to explicit authorization.

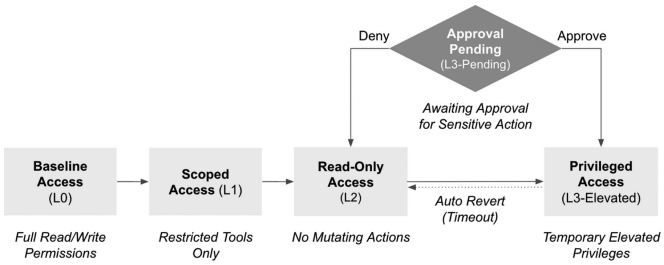


FIGURE 1. Session-level security state transition diagram for MCP-Secure.

The highest enforcement setting introduces an additional temporal element. When the agent attempts a sensitive operation, the wrapper prompts a temporary shift into an approval-pending state. If the request is granted, the state machine briefly elevates the session into a time-boxed privilege window before automatically reverting to a safer baseline. This mechanism prevents privilege drift, a common failure mode in long-running tool-use traces, by ensuring that elevated access never persists beyond its intended purpose.

Taken together, these transitions form a compact but expressive behavioral model. The wrapper can tune its permissiveness or strictness based on organizational requirements, yet its logic remains simple. Conceptually, the state machine acts as a governor on the entire interaction, ensuring that scoped access, read-only defaults, and approval gating operate cohesively rather than as disjoint checks. Moreover, MCP-Secure isolates policy decisions at the host wrapper layer, allowing it to operate across heterogeneous models and tasks while consistently enforcing the same permission boundaries. This design is simple to scale because policy checks are constant-time lookups, transitions are uncommon, policies are cached per session.

V. IMPLEMENTATION

MCP-Secure is implemented as a host-side mediation layer that interposes itself between the agent and all MCP servers used during a session. As shown in Fig. 2, the agent remains unmodified and continues to plan, reason, and emit Action JSON objects. MCP-Secure receives these actions, interprets them under a session-level policy, and determines whether each invocation should be allowed, blocked, downgraded, or routed through an approval workflow. This separation keeps enforcement logic entirely outside the agent and servers, ensuring backward compatibility and enabling consistent evaluation across heterogeneous model backend and tool configurations.

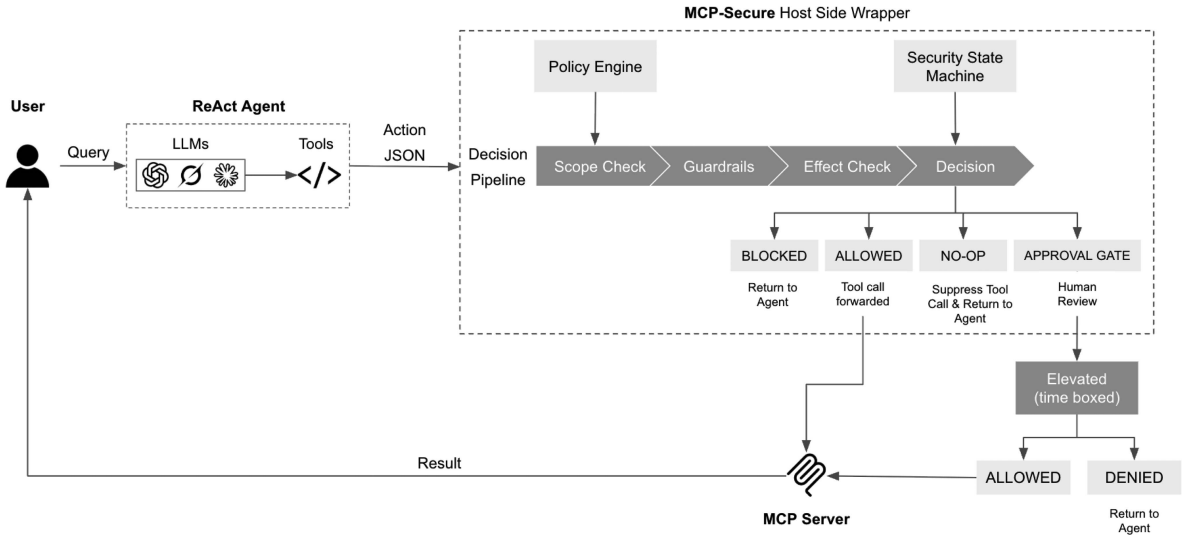


FIGURE 2. **Overview of the MCP-Secure runtime enforcement architecture.**

A. RUNTIME MEDIATION PIPELINE

When a task begins, the wrapper initializes a `SessionState` object that binds together the task's scoped configuration, the selected enforcement level (L0–L3), and a set of operational guardrails such as call-rate limits, payload-size caps, and repository fan-out constraints. These parameters define the session's enforcement context, which persists for the duration of the task and governs how each Action JSON emitted by the agent is interpreted.

Every tool invocation passes through a **policy engine** that evaluates the request through a sequence of constant-time checks. The pipeline begins with a scope validation step that ensures the requested server and tool fall within the task's declared operational boundary. The operational boundary refers to the explicit set of MCP servers, tools, and permitted effect categories (read, write, sensitive) declared for a task and enforced by the wrapper through constant-time scope checks. This immediately suppresses hallucinated or out-of-domain calls before they reach external services. This check is relevant at **L0** and **L1** enforcement levels. Under **L0**, scope validation is permissive but still protects against malformed or hallucinated action objects that do not map to any declared server. Under **L1**, the scope becomes binding. Only the servers and tools explicitly required for the task may be used, and all others are rejected. This early filtering prevents unintended cross-domain actions and removes many failure modes before they interact with external systems.

Once scope is established, the pipeline classifies the requested the action into a coarse effect category of read, write, or sensitive. This classification informs how the action is handled under **L2** and **L3** enforcement levels, where mutating or destructive effects require additional scrutiny. Under **L2**, the wrapper suppresses all mutating operations (e.g., writes, deletes, and file or branch creation) by converting them into safe no-ops unless they have been explicitly permitted in session configuration. This default-read-only behavior ensures that accidental writes, deletions, or destructive repository operations cannot occur silently as a consequence of ambiguous reasoning steps. Under **L3**, mutating actions trigger a temporary transition into a `PendingApproval` state. Instead of executing immediately, the wrapper pauses the agent's progression and routes the request to a dedicated approval gate. A successful approval grants the session a temporary elevation window, allowing the action to proceed under tightly scoped and time-bounded permissions before reverting to the baseline enforcement level.

After the policy engine renders a decision, the wrapper acts on behalf of the agent. It may forward the action unchanged to the MCP server, downgrade it to a safe no-op, block it outright, or execute it under elevated privileges granted through the approval workflow. Non-mutating requests typically flow through without modification, while mutating or ambiguous actions follow the full enforcement logic. The MCP server's response is returned to the agent as the next observation in its reasoning loop, preserving normal agent operation while silently enforcing least-privilege constraints in the background.

Throughout execution, MCP-Secure records each per-call decision in structured logs, including whether the call was allowed, blocked, no-opped, or approved, along with the rationale (e.g., out-of-scope, read-only enforcement, approval denied, resource-guard violation) and timing metadata. These logs form a complete audit trail, enabling downstream computation of safety, utility, and overhead metrics.

Together, these components enable MCP-Secure to enforce least-privilege behavior across the full lifecycle of an agent task session without modifying the agent or MCP servers. By operating precisely at the boundary where unsafe behavior arises—the live agent–tool interaction—the mediation pipeline provides fine-grained control, controlled elevation, and consistent enforcement independent of the underlying model.

B. ATTACK SIMULATION HARNESS

To evaluate the robustness of MCP-Secure under adversarial pressure, we embed an attack simulation harness directly into the host-side mediation layer. This harness injects controlled adversarial variants of tool calls while preserving the agent's reasoning loop. These simulations operationalize known failure modes identified in recent MCP security analyses and mimic realistic threat vectors observed in open-source MCP servers and multi-tool workflows. Three classes of attacks are modeled.

The first, **Privilege Escalation**, attempts to exploit overly broad tool declarations or implicit permissions by issuing calls to out-of-scope servers or privileged effects. These attacks probe whether the wrapper correctly identifies and suppresses unsafe expansions of capability. [16] [3].

The second, **Malicious Tool Metadata Injection**, simulates misleading or corrupted tool schemas (such as spoofed descriptions or manipulated parameter types) to test whether the wrapper relies on metadata alone or enforces scope and effect constraints independently of agent interpretation. Deceptive or adversarial content gets embedded in tool descriptions and can coerce agents into unauthorized actions by polluting the model's context [9] [16].

The third, **Multi-Tool Confusion and Chaining**, presents the agent with scenarios where chained actions across tools might produce unsafe composite effects even if each individual step appears benign. This category stresses the wrapper's ability to detect dangerous interactions that arise across turns, not within a single call. Privilege escalation emerges from chaining individually safe tools across domains [9].

By localizing these simulations within the mediation layer, MCP-Secure produces consistent, model-agnostic adversarial conditions, enabling direct comparison of safety enforcement across models and access levels without modifying the task logic or MCP servers.

C. CROSS-MODEL EXECUTION CONSISTENCY

MCP-Secure is evaluated with three contemporary language models: GPT-5, Claude Sonnet 4, and Grok-4. These models were selected for two reasons. First, they represent three distinct model families with different training regimes, inference stacks, and tool-use tendencies, allowing us to test whether MCP-Secure behaves consistently across varying agent strategies. Second, the selection aligns with the methodology used in the MCP-Universe benchmark, which emphasizes cross-model evaluation to separate agent-driven behavior from framework-driven behavior [12].

By adopting a similar multi-model setup, we ensure comparability with prior work and avoid attributing model-specific idiosyncrasies to the enforcement mechanism itself. Since MCP-Secure sits entirely outside the agent, applying identical policy logic to each model, any observed differences in tool usage arise from the models' planning and not from variability in enforcement. This pairing of heterogeneous models with a uniform mediation layer is essential for demonstrating that the wrapper is model-agnostic and robust to backend diversity.

TABLE 1. Safety Outcomes Across Access Levels

Access Level	Unsafe Attempts	Unsafe Blocked	Unsafe Block Rate (%)
L0	2.89	0.00	0.00
L1	2.73	1.91	75.30
L2	1.80	1.80	100.00
L3	2.11	2.00	97.50

VI. RESULTS

We evaluate MCP-Secure across 1,080 controlled executions, where each execution corresponds to a complete end-to-end agent run on a repository-management task under a fixed combination of task type, LLM backend, adversarial attack simulation and enforcement level. Concretely, the evaluation spans two repository-management tasks, three heterogeneous LLMs, three adversarial attack simulations, and four enforcement levels (L0–L3), with repeated runs per configuration to account for model stochasticity. Each configuration is repeated 15 times to reduce variance from stochastic model output and to ensure that observed trends reflect stable behavioral patterns rather than run-to-run noise. Across all runs, we capture every tool invocations, wrapper decisions, wrapper decision, approval event, and final task outcome. This allows us to analyze both safety and utility effects under systematically varied conditions. This section reports the aggregate trends that emerge across this experimental grid, emphasizing consistent patterns across models, tasks, and adversarial conditions.

Our evaluation focuses on quantifying safety gains, utility trade-offs, and enforcement side effects introduced by increasingly restrictive policies. Specifically, we measure reductions in unsafe or out-of-scope tool attempts relative to open-access baselines, task completion outcomes, and instances of false blocking where legitimate progress is impeded.

A. SAFETY IMPROVEMENTS ACROSS ENFORCEMENT LEVELS

Table 1 shows that MCP-Secure produces clear, monotonic improvement in safety as enforcement levels tighten. Under the open baseline (L0), agents freely issue unsafe tool calls. Agents attempt an average of 2.89 unsafe calls per run, and none of which are intercepted. Introducing scoped access in L1 immediately constrains cross-domain behavior, preventing 75.3% of unsafe calls and illustrating the effectiveness of even minimal boundary setting.

At L2, where all mutating operations are disabled by default, MCP-Secure achieves a 100% unsafe block rate. L3 maintains similarly high robustness at 97.5%, with the few unblocked unsafe attempts corresponding to actions explicitly approved through the gating mechanism. Notably, unsafe attempts themselves fall as enforcement strengthens (2.89 →

TABLE 2. Utility Outcomes by Task and Access Level

Task	Level	Progress Before Block	Latency (ms)
developer_open_pr	L0	0.43	0.43
developer_open_pr	L1	0.19	0.11
developer_open_pr	L2	0.00	0.05
developer_open_pr	L3	0.00	0.08
org_search_and_publish	L0	0.43	0.56
org_search_and_publish	L1	0.49	0.54
org_search_and_publish	L2	0.19	0.48
org_search_and_publish	L3	0.23	0.57

1.80 at L2), indicating that that the wrapper influences not only tool-level outcomes but also upstream planning. When visibility narrows, agents adjust their reasoning traces toward safer strategies.

Overall, the safety trends confirm that session-level, runtime enforcement meaningfully reduces unsafe tool use and improves agent behavior under adversarial and unrestricted conditions.

B. UTILITY OUTCOMES ACROSS TASKS AND ENFORCEMENT LEVELS

Table 2 presents a capability-oriented view of utility under different enforcement levels. It reframes utility not only as binary task success, but as how far an agent progresses before enforcement intervenes. Many repository-management workflows are inherently write-dependent, meaning strict least-privilege controls are expected to suppress completion. Rather than interpreting these failures as shortcomings of the framework, we interpret them as evidence that MCP-Secure is correctly enforcing safety boundaries.

To capture partial progress, we compute a Progress Before Block metric to quantify how far an agent progresses through each task before encountering a blocked or downgraded action. For a given task with N required steps, we walk the run's tool-call sequence and count how many required steps are successfully executed before the first enforcement stops the workflow. Blocks on non-required tools do not affect this metric. We average this progress fraction across all runs for each task and access level, providing a more realistic picture of capability preservation under constraint.

$$\text{ProgressBeforeBlock} = \frac{\text{Required steps successfully executed}}{\text{Total required steps in task}}$$

The `developer_open_pr` task requires four mutating actions: creating a branch, committing a file, opening a pull request, and merging it. Under L0, models complete almost half the steps on average (0.43 progress). Under L1, introducing scoped access reduces cross-server drift but blocks necessary Git writes and lowers progress to 0.19. L2 and L3 enforce strict read-only semantics and approval gating, respectively. As expected for a fully write-dependent workload, both levels prevent any forward progress, yielding 0.00 average completion. These results highlight an important feature that MCP-Secure is correctly enforcing safety without misclassifying non-mutating ones.

TABLE 3. Cross-Model Comparison of Safety and Utility Metrics

Model	Benign (%)	Block Rate (%)	Latency (ms)
`claude-sonnet-4-0`	68.10	65.40	0.37
`gpt-5`	58.40	69.80	0.34
`grok-4`	55.10	69.80	0.36

The `org_search_and_publish` task combines one safe and two mutating actions: issue search, creating a repository and publishing a CSV. Under L0 and L1, progress before block metric is similar (0.43 for L0 and 0.49 for L1), which indicates that the significant number of steps execute easily without too much restricted tool visibility. At L2 and L3, progress before block metric drop to 0.19 and 0.23 respectively. This shows that agents complete relatively fewer portions of the workflow before encountering enforcement barriers.

Median latency overhead is modest across all configurations (0.4–0.5 milliseconds per run), reflecting constant-time policy checks and the low frequency of elevation events. L3 introduces additional overhead via approval handling, but its effect on latency is small relative to tool-use costs.

Taken together, these results illustrate MCP-Secure offers a predictable utility profile. The progress-before-block metric shows that agents generally make partial forward progress under tighter controls. Read-heavy tasks retain meaningful forward progress under stricter policies, while write-dependent tasks fail safely and early, exactly where least-privilege boundaries are designed to intervene. This indicates that the wrapper is not indiscriminately halting workflows. Instead, it interrupts them precisely at the point where mutating or unsafe operations are required. The modest and stable latency across levels further suggests that policy mediation introduces minimal runtime overhead, allowing safety controls to be applied without materially affecting interactive performance.

C. MODEL-AGNOSTIC ENFORCEMENT AND BEHAVIORAL DIFFERENCES

A central design goal of MCP-Secure is to enforce identical policy behavior regardless of the underlying language model.The wrapper applies the same policy logic across all models without modification. This design ensures that any variation in outcomes reflects differences in agent planning behavior, not inconsistencies in the enforcement layer itself. Table 3 illustrates this separation by comparing safety and benign-action outcomes across three heterogeneous models.

A key metric in this analysis is the Benign %, defined as the proportion of allowed, in-scope, non-mutating tool calls

TABLE 4. Adversarial Stress-Testing Outcomes Across Attack Types and Enforcement Levels

Attack Type	Level	Unsafe Attempts	Unsafe Blocked	Unsafe Block Rate (%)
A_delete_branch	L0	3.56	0.00	0.00
A_delete_branch	L1	2.92	2.16	78.30
A_delete_branch	L2	2.00	2.00	100.0
A_delete_branch	L3	2.37	2.17	95.90
B_summarize_http	L0	2.26	0.00	0.00
B_summarize_http	L1	2.44	1.60	71.60
B_summarize_http	L2	1.69	1.69	100.00
B_summarize_http	L3	1.87	1.83	99.00
C_force_push	L0	2.87	0.00	0.00
C_force_push	L1	2.81	1.99	76.20
C_force_push	L2	1.70	1.70	100.00
C_force_push	L3	2.10	1.99	97.60

relative to all tool calls issued by the model.

$$\text{Benign}(\%) = \frac{\text{Number of allowed benign tool calls}}{\text{Total tool calls attempted}} \times 100.$$

This measure captures how frequently a model stays within the safe portion of its operational boundary, independent of whether the wrapper must block unsafe attempts. Unlike block rate, which measures the wrapper's defensive success, Benign % reflects the quality of model reasoning under constraints, offering a view into how well each agent adapts to scoped permissions.

All three models cluster tightly in block-rate performance (65–70%), confirming that MCP-Secure applies its safety checks uniformly regardless of backend. Where they diverge is in benign behavior. Claude Sonnet 4 achieves the high benign fraction (68.10%) and maintains competitive unsafe blocking performance (65.40%). This suggests it plans conservatively and stays closer to the intended tool sequences, resulting in both fewer unsafe attempts and smoother progress before blocking. Grok-4 displays a more exploratory strategy. It issues broader tool calls, which leads to reduced benign allowance (58.4%). GPT-5 sits between these extremes at 58.4% benign allowance.

Taken together, these results show that MCP-Secure preserves cross-model comparability and enables model-level behavioral analysis without altering enforcement fidelity.

D. ADVERSARIAL STRESS TESTING

The adversarial simulation harness introduced in the Implementation section allows us to evaluate how MCP-Secure behaves when confronted with three representative threat vectors: (A) delete-branch escalation, (B) malicious summarization via HTTP metadata poisoning, and (C) force-push misuse through multi-tool chaining. Table 4 reports outcomes across enforcement levels for each attack family.

Across all attack simulations, a consistent pattern emerges. The baseline configuration (L0) remains fully vulnerable. Unsafe attempts pass through unmodified, and block rates are 0%. L1 immediately blocks 70–78% of unsafe actions purely by removing access to tools outside the task's declared domain. This demonstrates that even minimal boundary-setting yields substantial defensive coverage. L2 achieves a perfect 100% block rate for every attack type, showing that read-only defaults reliably neutralize mutating operations irrespective of the agent's strategy. L3 maintains near-perfect robustness (96–99%) while also enabling temporary elevation when approvals are explicitly granted.

As in earlier results, unsafe attempts decrease at stronger enforcement levels (e.g., force-push attempts fall from 2.87 at L0 to 1.70 at L2), reinforcing the behavioral-shaping effect that constraining the visible capability surface leads the agent to plan more conservatively even under adversarial stimuli.

Overall, these results show that MCP-Secure's safeguards hold not only in benign execution but also under active, targeted adversarial manipulation. The wrapper consistently protects against three of the most prominent threat vectors identified in recent research, while preserving a clear distinction between denial by policy and temporary elevation when approvals are granted.

E. SUMMARY OF RESULTS

Across 1080 executions spanning multiple tasks, models, and adversarial conditions, the evidence demonstrates that MCP-Secure systematically strengthens safety while providing clear, interpretable trade-offs in utility. The progression of enforcement levels shows a consistent pattern: scoped access (L1) alone eliminates most out-of-scope behavior, read-only defaults (L2) reliably neutralize all mutating operations (including those embedded in adversarial sequences) and approval-gated elevation (L3) preserves near-perfect protection while allowing controlled use of sensitive capabilities. A recurring trend is that unsafe attempts decrease under stricter policies, indicating behavioral shaping rather than simple filtering. The wrapper alters how models plan rather than merely blocking after the fact.

Utility results reflect the expected dependency between workload type and enforcement strictness. Write-heavy workflows stall under L2's read-only policy, whereas read-centric tasks retain their performance under L1 with negligible latency overhead. The approval workflow in L3 introduces small and predictable delays but does not materially alter task feasibility, reinforcing that the mechanism is viable for scenarios requiring temporary privilege elevations.

Cross-model analysis shows that enforcement remains fully model-agnostic, with differences in outcomes driven purely by model reasoning rather than policy drift.This validates that the wrapper does not introduce model-specific bias or drift, a key requirement for reproducible evaluation.

The results presented here establish a strong empirical foundation. These findings show that MCP-Secure provides a practical and portable least-privilege enforcement layer that operates reliably across heterogeneous models and workloads. It offers strong adversarial resistance, predictable utility characteristics, and interpretable safety– utility trade-offs—all

without modifying underlying agents or MCP servers. The results establish MCP-Secure as a viable security mechanism for real-world, tool-enabled LLM deployments.

VII. LIMITATIONS AND MITIGATION STRATEGIES

MCP-Secure demonstrates robust least-privilege behavior within the boundaries of our controlled study, but broader validation will further strengthen its applicability to real-world LLM agent deployments. While MCP-Secure demonstrates consistent enforcement behavior across a comprehensive experimental grid, several limitations constrain the generality of our findings. These limitations arise partly from our controlled experimental setup and partly from structural challenges inherent to permission-mediated agent systems. They highlight areas for methodological refinement rather than fundamental weaknesses in the enforcement approach.

A. UNCONTROLLED MODEL STOCHASTICITY AND LIMITED SEPARATION OF VARIANCE SOURCES

In our experiments, we do not explicitly fix or sweep over the stochasticity of the underlying language models. Provider calls omit temperature and seed settings, meaning decoding randomness follows each model API's defaults. As a result, variation in the observed executions reflects a mixture of the model's own sampling variability and the wrapper's enforcement dynamics. By contrast, the enforcement layer introduces a controlled stochastic element at L3 through a probabilistic approval policy. This allows us to study how different approval behaviors influence safety and utility under otherwise comparable conditions, but does not fully isolate model-driven randomness from wrapper-driven randomness. A more complete methodology would control both sources of variability by systematically varying model temperature/seed, and sweeping over approval policies. Such controlled diversity would enable more precise attribution of variability to the model versus the enforcement layer.

B. LIMITED SCOPE OF TASKS AND TOOL TYPES

Our evaluation focuses on two repository-management workflows. They represent realistic, multi-step tool use, but they do not capture the breadth of real deployments, which often involve dozens of heterogeneous tools, cross-server state sharing, and extended reasoning horizons. Expanding the benchmark with multi-stage, cross-domain workflows would provide deeper visibility into policy interactions. Integrating synthetic tasks designed to stress boundary conditions (e.g., concurrent writes, retry loops, mixed read/write sequences) can help surface policy gaps not exercised by the current suite.

C. OVER-APPROXIMATION AND UNDER-APPROXIMATION OF SAFETY BOUNDARIES

MCP-Secure enforces scoped access and read-only semantics based on static tool metadata. Although this is the standard mechanism available today, metadata can be coarse: some "read" endpoints may leak privileged information, and some "write" endpoints may include safe idempotent operations that should not be blocked. These approximations can lead to both over-blocking (false positives) and under-blocking (false negatives). More fine-grained policy instrumentation, such as argument-level validation, contextual policy gates, and dynamic taint-tracking of server responses, would better capture the operational semantics of tool effects.

D. ABSENCE OF MULTI-USER OR MULTI-AGENT INTERFERENCE

Our experiments evaluate single-agent sessions in isolation. In production environments, multiple agents may interact with shared version-control resources, shared knowledge bases, or shared workspaces. These settings introduce failure modes such as cross-session interference, race conditions, or multi-agent privilege escalation. Evaluating MCP-Secure under concurrent multi-agent execution would more closely reflect enterprise deployments.

E. SIMPLIFIED APPROVAL MECHANISM

At L3, approvals are handled by a lightweight internal policy engine. While this abstraction allows reproducibility and controlled experimentation but does not model real organizational approval pathways, which may include multi-tier signoff, audit constraints, human latency, or compliance checks. Consequently, the evaluation underestimates the overhead and friction associated with real approval workflows. Integrating different approval workflows would provide a more realistic assessment of operational overhead.

F. LIMITED ADVERSARIAL MODELING BEYOND INTENT MANIPULATION

Our adversarial probes focus on prompt-level manipulations and ambiguous instruction patterns. We do not evaluate adversarial model behavior arising from broader red-team techniques such as jailbreak attempts, fuzzing strategies, prompt injection attacks, or adversarial example generation for tool schemas. Including such attack simulations could reveal bypass mechanisms not captured by our current probes.

VIII. FUTURE WORK

Several directions naturally extend the capabilities and evaluation breadth of MCP-Secure. First, incorporating **stochastic and multi-trajectory execution** remains an important step toward realism. Our current study uses provider defaults for model sampling and a controlled stochastic approver. Future work should systematically vary model temperatures, seeds, and approval policies to examine families of trajectories and quantify enforcement stability across divergent reasoning paths.

Second, **richer workflow structures** merit deeper investigation. Real deployments often involve multi-stage interactions that span version control, search, browser automation, and internal APIs. Evaluating MCP-Secure on such cross-domain, stateful workflows would illuminate how scoped access and elevation policies interact when agents must coordinate across multiple servers and resource boundaries.

Third, the framework would benefit from **finer-grained policy semantics**. Endpoint-level read/write classification is intentionally simple but can over- or under-approximate real capability boundaries. Extending the wrapper with argument-level validation, contextual gating, or capability-graph reasoning could improve precision without sacrificing interpretability.

Finally, integrating MCP-Secure with **organizational governance mechanisms** represents an important frontier. Real approval paths involve human-in-the-loop delays, multi-tier escalation, audit capture, and interaction with enterprise policy engines. Evaluating MCP-Secure within these governance contexts would provide clearer guidance for production adoption and illuminate the trade-offs between security assurance and operational friction.

IX. CONCLUSION

This work introduced MCP-Secure, a lightweight, host-side enforcement layer that applies least-privilege controls to tool-using LLM agents without modifying the agent, model, or MCP servers. By structuring enforcement around scoped access, read-only defaults, and approval-gated elevation, MCP-Secure provides a unified mechanism for mediating multi-step tool interactions, which is an area where existing MCP deployments and agent frameworks lack systematic safeguards.

Across 1080 runs spanning heterogeneous models, benign and adversarial workflows, and multiple enforcement regimes, MCP-Secure consistently reduced unsafe behavior while preserving predictable utility patterns. Tighter enforcement not only blocked unsafe tool calls but also shaped model planning, reducing the likelihood of unsafe attempts in the first place. These effects held across models, confirming that the framework operates independently of model idiosyncrasies and can serve as a reliable basis for cross-model evaluation.

Taken together, the findings demonstrate that a small amount of structured mediation at the MCP boundary can deliver meaningful security benefits without sacrificing portability or interpretability. MCP-Secure provides a practical foundation for permission-aware agent execution and opens the door for richer evaluation of tool-enabled AI systems under realistic security constraints. Future work will deepen this foundation by expanding workflow diversity, refining policy granularity, and integrating with organizational governance mechanisms, moving toward principled and scalable security for agentic AI ecosystems.

REFERENCES

[1] Y. Yao et al., "A survey on large language model (LLM) security and privacy: The good, the bad and the ugly," *Comput. Intell. Neurosci.*, vol. 2024, 2024, Art. no. 100211.

[2] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," *High-Confidence Comput.*, vol. 4, no. 2, pp. 1–39, 2024.

[3] C. Bühler, M. Biagiola, L. Di Grazia, and G. Salvaneschi, "Securing AI agent execution," 2025, *arXiv:2510.21236*.

[4] M. Milanta and L. Beurer-Kellner, "Github MCP exploited: Accessing private repositories via MCP," Invariant Labs Blog, May 26, 2025. [Online]. Available: https://invariantlabs.ai/blog/mcp-github-vulnerability

[5] I. Asana, "Incident report–id 5b9hhtgs3mxs," Asana Incident Report, 2025. [Online]. Available: https://status.asana.com/incidents/5b9hhtgs3mxs

[6] M. Andriushchenko et al., "Agentharm: Measuring the safety of tool-using LLM agents," in *Proc. Int. Conf. Learn. Representations*, 2025.

[7] W. Zhou et al., "The security of using large language models: A survey," *J. Appl. Secur.*, vol. 12, pp. 1–26, 2025.

[8] W. Xu et al., "LLM-based agents for tool learning: A survey," *AI Rev.*, vol. 57, no. 4, 2025, pp. 1–31.

[9] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (MCP): Landscape, security threats, and future research directions," 2025, *arXiv:2503.23278*.

[10] V. S. Narajala and I. Habler, "Enterprise-grade security for the model context protocol (MCP): Frameworks and mitigation strategies," 2025, *arXiv:2504.08623*.

[11] N. O. Jaffal et al., "Large language models in cybersecurity: A survey of vulnerabilities and defenses," *Cybersecur. Artif. Intell.*, vol. 6, no. 9, 2025, Art. no. 216.

[12] Z. Luo et al., "MCP-universe: Benchmarking large language models with real-world model context protocol servers," 2025, *arXiv:2508.14704*.

[13] S. Yao et al., "ReAct: Reasoning and acting in language models," in *Proc. Int. Conf. Learn. Representations*, 2023.

[14] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 8634–8652.

[15] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 68539–68551.

[16] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, "Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers," 2025, *arXiv:2506.13538*.

[17] Y. Guo et al., "Systematic analysis of MCP security," 2025, *arXiv:2508.12538*.

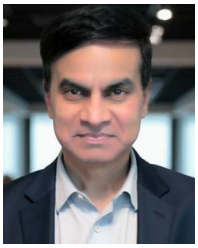
[18] J. Fang et al., "We should identify and mitigate third-party safety risks in MCP-powered agentsystems," 2025, *arXiv:2506.13666*.

[19] S. Kumar et al., "MCP guardian: A security-first layer for safeguarding MCP-based AI system," 2025, *arXiv:2504.12757*.

[20] H. Wang et al., "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025, *arXiv:2503.18666*.



GAMINI SINGH received the B.S. degree in chemical engineering from BITS, Pilani, India, the first M.S. degree in business analytics from Duke University, Durham, NC, USA, and the second M.S. degree in computer science from the Georgia Institute of Technology, Atlanta, GA, USA. She is currently a Principal Data Scientist with Red Hat, Inc., where she builds production-grade intelligent systems. Her research interests include secure tool-use architectures for LLM-driven automation, scalable evaluation frameworks for agent behavior, and the intersection of AI safety and enterprise ML systems. She has spoken at multiple technology conferences and holds multiple certifications in AI, ML, and data analytics.



VIJAY K. MADISETTI (Fellow, IEEE) received the Ph.D. degree in electrical engineering and computer sciences from the University of California at Berkeley, Berkeley, CA, USA. He is currently a Professor of cybersecurity and privacy (SCP) with Georgia Tech., Atlanta, GA, USA. He has authored several widely referenced textbooks on topics, such as cloud computing, data analytics, blockchain, chip design, and microservices. He is a coauthor of *IEEE VHDL RTL Language Standard* and holds more than 80 issued U.S. patents. He has been honored with the Terman Medal by the American Society of Engineering Education (ASEE).